

Unixによるテキストファイル処理

2026/2/5

2026/02/05

作業場所

- 以降の作業は、以下のディレクトリで行います。

```
~/gitc/data/7_text/
```

cd コマンドを用いてディレクトリを移動し、

pwd コマンドを利用して、カレントディレクトリが上記になっていることを確認してください。

2026/2/5

コマンド復習

- `wc` [ファイル名]
 - ファイルの行数、単語数、文字数を出力する
- `head` [-行数] [ファイル名]
 - ファイルの先頭から指定した行数(指定しないと10行)を出力する
- `tail` [-行数] [ファイル名]
 - ファイルの最後から指定した行数(指定しないと10行)を出力する
- `less` ファイル名
 - ファイルの内容を閲覧する

2026/2/5

本講で扱うテキスト処理コマンド

- `grep` 正規表現パターンの検索
- `sed` 文字列置換等によるファイルの変換
- `awk` 様々なテキストファイル処理

2026/2/5

正規表現による文字列検索 (grep)

- `grep 'パターン' [ファイル名 ...]`
 - ファイル中でパターンを含む行を出力する
- 例) `grep 'GO' 1433T_HUMAN.sprot`
- 1433B_HUMAN.sprot から GO を含む行を検索する。

```
...  
DR Genevestigator; P27348; -.  
DR GermOnline; ENSG00000134308; Homo sapiens.  
DR GO; GO:0005813; C:centrosome; IDA:HPA.  
DR GO; GO:0005634; C:nucleus; IDA:HPA.  
DR Bgee; P27348; -.  
....
```



```
DR GO; GO:0005813; C:centrosome; IDA:HPA.  
DR GO; GO:0005634; C:nucleus; IDA:HPA.
```

2026/2/5

正規表現による文字列検索 (grep)

- `grep 'パターン' [ファイル名 ...]`
 - ファイル中でパターンを含む行を出力する
- 例) `grep 'GO' 1433T_HUMAN.sprot`
- 1433B_HUMAN.sprot から GO を含む行を検索する。
- 例) `grep '^FT' 1433T_HUMAN.sprot`
- 1433B_HUMAN.sprot から FT で始まる行を検索する。
- `grep -v` パターンを含まない行を出力する。
 - `grep -i` 大文字小文字を区別しない。
 - `grep -w` パターンを単語としてマッチ
 - ファイル名は複数指定可能
 - ファイル名を省略すると、標準入力から文字列を読み込んでパターンを検索する

2026/2/5

正規表現

grepは「正規表現」によってパターンを指定し、照合したい文字列集合を規定する

- 通常の文字列はそのまま表現される
 - 例) `File1` (`File1`にマッチ)
- 特殊な意味を持つ文字(メタキャラクタ)によって規則を表現
 - 例) `[]`は文字集合を規定する
`File[1-3]` (`File1`, `File2`, `File3` のいずれにもマッチ)
- `\`によってメタキャラクタの特殊な意味を打ち消せる
 - 例) `\[abc\]` (`[abc]` という文字列にマッチ)

2026/2/5
注意) 正規表現にはシェルのメタキャラクタが含まれるので、そのままコマンドラインで指定すると思わぬエラーになることが多い。そこで、パターンは `' '` で囲むようにする。

正規表現(一部)

- **.(ドット) 任意の1文字**
 - 例) `a.c`
`abc`, `adc` など、`a`と`c`の間に任意の1文字を含む文字列にマッチ
- **[](角形括弧) 文字の集合**
 - 例) `[ATGC]`
`A`, `T`, `G`, `C`のいずれかにマッチ
 - 例) `[a-d]`
`a`, `b`, `c`, `d`のいずれかにマッチ
 - 例) `[^abd]`
`a`, `b`, `d` 以外のいずれかにマッチ
- **^ 行の先頭 \$ 行の終端**
 - 例) `^ID`
行の先頭が`ID`である行とマッチ
- *** 0回以上の繰り返し**
 - 例) `a.*m`
`a`と`m`の間に任意の文字列を含む(`am`, `arm`, `alarm`, `am am`など)

演習 (grep)

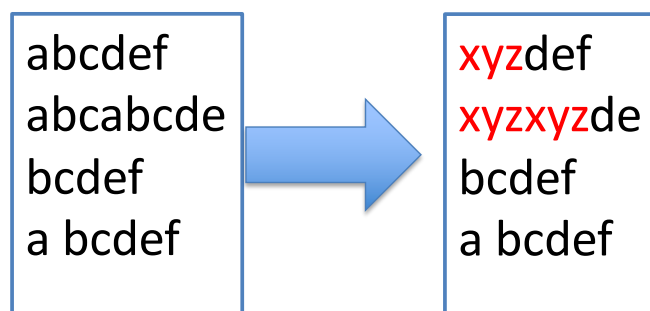
- `ecoli.sam` ファイルから、`grep` (`egrep`) コマンドを用いてヘッダ行(行頭に@を含む)を表示せよ。

2026/2/5

文字の置換など (sed)

```
sed 's/置換対象パターン/置換文字列/g' [ファイル名]
```

- ファイル中の、指定した正規表現パターンに合致するすべての文字列を、指定した置換文字列で置き換える。置換文字列が空の場合は文字列の削除になる。
 - 例) `sed 's/abc/xyz/g' abcdef.txt`
ファイル中の文字列`abc`をすべて`xyz`に置き換える。



2026/2/5

文字の置換など (sed)

```
sed 's/置換対象パターン/置換文字列/g' [ファイル名]
```

- ファイル中の、指定した正規表現パターンに合致するすべての文字列を、指定した置換文字列で置き換える。置換文字列が空の場合は文字列の削除になる。
 - 例) `sed 's/abc/xyz/g' file`
file中の文字列abcをすべてxyzに置き換える。
- 最後のgをつけない場合は、各行で最初にマッチしたパターンのみが置換される。
 - 例) `sed 's/:/ /' file`
各行で最初に出現した : をスペースに置き換える

2026/2/5 sedコマンドにも一般にシェルのメタキャラクタが含まれるので、パターンの指定は常に' 'で囲むようにする。

FASTA形式ファイルの処理例

mouse.fas というFASTAファイルからタイトル行を抜き出し①、行頭の'>'を除去して②、最初のスペースをタブに換える③。

```
grep '^>' mouse.fas | sed 's/^> //' | sed 's/ /¥t/' > mouse.tit
```

①

②

③

```
> mouse:Q0VBH3 Olfr629 protein
MAGNATHHIASFFLVGIPGLENFHCWIGIPVCLLFALTLLGNSIILTTVKLEPSLHQPMY
FFLCMLAMNDMCLTCSTALKMLGIFWFDEHWINFDAQLTQMFFIHTLCIMESAILVAMAF
DRFVAICIPHYTSILTT¥tKIGLVGLSRAILMIMPCPLLKRLLYYTKYVIHHAYCE
HMAVVKMASGNTQVNRIY¥tALSVTIFDLGLIVTSYIKILQAVFRLSSQNARSKALGT
CVAHVCTILAFYTPALFSFLTHRFGKNVPASIHIIIFAILYLLVSPTVNPLVYGAKTKQIR
DRVVSLFLFSQKQKF
> mouse:Q9JMI7 Testis-expressed protein 101
MGACRIQYVLLIFLLIASRWTLVQNTYQCVSQTLSLEDDPGRTFNWTSKAEQCNPGELCQ
ETVLLIKADGTRTVVLASKSCVSQGGAEVTFIQYTAPPGLVAISYSNYCNDSLCNNKDSL
ASVWRVPETTATSNMSG¥tPTCVALGSCSSAPSMPCANGTTQCYQGRLEFSGGGMDAT
VQVKGCTTTIGCRLMAM¥tGPMTVKETCSYQSFLQPRKAEIGASQMPTSLWVLELLFP
LLLLPLTHFP
```

¥t(タブ記号)は
Control + vを
押した後に
タブキーで
入力できます。

```
mouse:Q0VBH3 Olfr629 protein
mouse:Q9JMI7 Testis-expressed protein 101
```

FASTA形式ファイルの処理例

mouse.fas というFASTAファイルからタイトル行を抜き出し①、
行頭の'>'を除去して②、最初のスペースをタブに換える③。

```
grep '^>' mouse.fas | sed 's/^> //' | sed 's/ /\\t/' > mouse.tit
```

①

②

③

```
>mouse:Q0VBH3 Olfr629 protein
MAGNATHHIASFFLVGIPGLENFHCWIGIPVCLLFALTLLGNSIILTTVKLEPSLHQPMY
FFLCMLAMNDMCLTCSTALKMLGIFWFDEHWINF'DACLTQMFFIHTLCIMESAILVAMAF
DRFVAICIPHYTSILTTPMVIKIGLVGLSRAILMIMPCPLLIKRLLYYTKYVIHHAYCE
HMAVVKMASGNTQVNRIYGILVALSVTIFDLGLIVTSYIKILQAVFRLSSQNARSKALGT
CVAHVCTILAFYTPALFSFLTHRFGKNVPASIHIIIFAILYLLVSPTVNPLVYGAKTKQIR
DRVVSLLSQKQKF
>mouse:Q9JMI7 Testis-expressed protein 101
MGACRIQYVLLIFLLIASRWTLVQNTYCQVSQTLSEDDPGRTFNWTSKAEQCNPGECLCQ
ETVLLIKADGTRTVVLASKSCVSQGGAEVTFIQYTAPPGLVAISYSNYCNDSLCNNKDSL
ASVWRVPETTATSNMSGTRHCPTCVALGSCSSAPSMPCANGTTQCYQGRLEFSGGGMDAT
VQVKGCTTTIGCRLMAMIDSVGPMTVKETCSYQSFLQPRKAEIGASQMPTSLWVLELLFP
LLLLPLTHFP
```



```
mouse:Q0VBH3      Olfr629 protein
mouse:Q9JMI7      Testis-expressed protein 101
```

テキストファイルの処理(awk)

awk 'コマンド' ファイル

- テキストファイルを処理する多機能なコマンド
- コマンドの一般形式は
パターン {アクション}
パターンに指定した条件に合致した行について、アクションで指定した操作を行う。パターンを省略するとすべての行が対象になる。
- タブ区切りテキストなどテーブル形式のファイルでは、\$1, \$2, ... によって各フィールド(カラム)の値を参照できる。

テーブルデータの処理 (awk)

- テーブルカラムの抽出

```
awk '{print $3,$4,$5}' datafile
```

- 3, 4, 5カラム目を出力
- パターンが指定されていないのですべての行が出力される。

ecoli.gtf

```
...
chr  eschColi_K12_refSeq  stop_codon  253  255 ...
chr  eschColi_K12_refSeq  exon        190  255 ...
chr  eschColi_K12_refSeq  start_codon 337  339 ...
chr  eschColi_K12_refSeq  CDS         337  2796 ...
```



```
# awk '{print $3,$4,$5}' ecoli.gtf
...
stop_codon  253  255
exon        190  255
start_codon 337  339
CDS         337  2796
```

2026/2/5

テーブルデータの処理 (awk)

- 条件を指定したフィルタリング

```
awk '$4<200 {print}' datafile
```

- 4カラム目が200未満の行を出力
- 出力フィールドが指定されていないので行全体を出力

ecoli.gtf

```
...
chr  eschColi_K12_refSeq  stop_codon  253  255 ...
chr  eschColi_K12_refSeq  exon        190  255 ...
chr  eschColi_K12_refSeq  start_codon 337  339 ...
chr  eschColi_K12_refSeq  CDS         337  2796 ...
```



```
# awk '$4<200 {print}' ecoli.gtf
chr  eschColi_K12_refSeq  start_codon 190 192 ...
chr  eschColi_K12_refSeq  CDS         190 252 ...
chr  eschColi_K12_refSeq  exon        190 255 ...
```

2026/2/5

テーブルデータの処理 (awk)

- テーブルカラムの抽出

```
awk '{print $1,$2,$5}' datafile
```

- 1,2,5カラム目を出力
- パターンが指定されていないのですべての行が出力される。

- 条件を指定したフィルタリング

```
awk '$3<200 {print}' datafile
```

- 3カラム目が200未満の行を出力
- 出力フィールドが指定されていないので行全体を出力

- 複数の条件の指定

```
awk '$2~/target/ && $3<200{print}' datafile
```

- 2カラム目にtargetを含み、3カラム目が200以下の行を出力
- 変数~/パターン/ は正規表現の照合

2026/2/5 awk コマンドにも一般にシェルのメタキャラクタが含まれるので、常に''で囲むようにすると良い

演習 (awk)

- awkコマンドを用いて、sce-spo.blastout の中で、以下の条件に合う行または要素を出力せよ。

- 1) 各クエリ行のqueryとsubjectを出力する
- 2) e-valueが1.0e-5以上の行を出力する
- 3) 2)の条件を満たす行についてquery, subject, description のみを出力する。

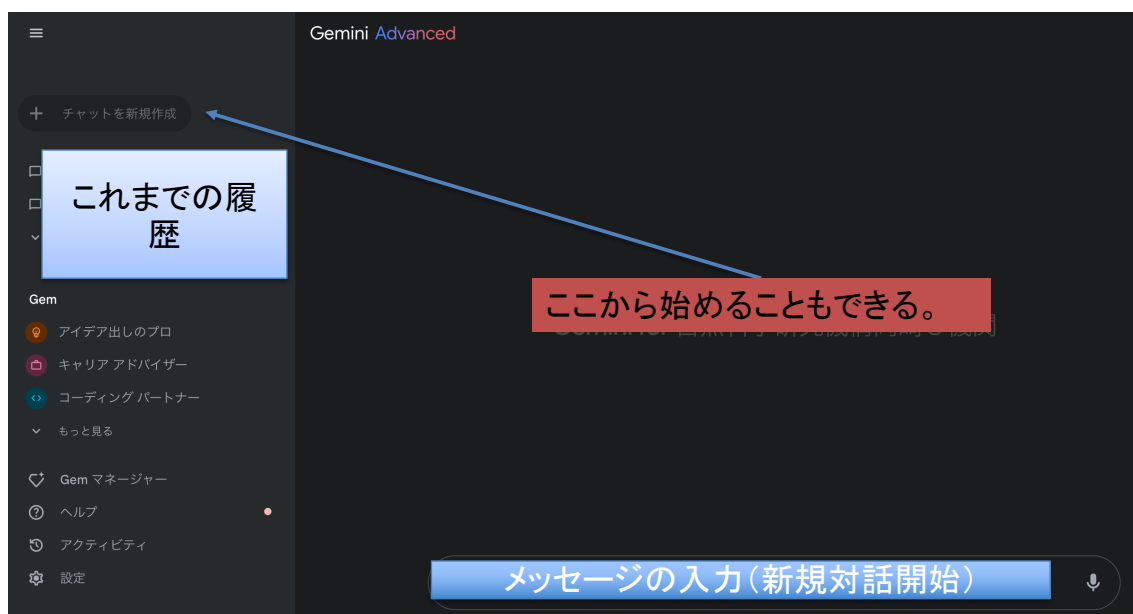
自分で考えてもよいですし、生成系AIを用いても構いません。

.blastout (tabular) の中身

query	subject	%identity	align-len	mismatch	gap_open	q_start	q_end	s_start	s_end	evalue	bit-score	description
spo:NP_001018179.1	sce:NP_010076.1	40.373	322	181	4	6	322	5	320	2.05e-81	248	hydroxymethylbilane synthase [Saccharomyces cerevisiae S288c]
spo:NP_001018181.1	sce:NP_014526.1	41.772	316	179	4	213	524	149	463	2.04e-80	265	non-canonical poly(A) polymerase PAP2 [Saccharomyces cerevisiae S288c]
spo:NP_001018181.1	sce:NP_014100.2	34.541	414	239	9	138	545	89	476	8.12e-72	243	non-canonical poly(A) polymerase TRF5 [Saccharomyces cerevisiae S288c]
spo:NP_001018181.1	sce:NP_012927.3	31.818	66	41	2	282	346	84	146	0.006	37.4	polynucleotide adenyllyltransferase PAP1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_013081.1	47.176	301	132	6	345	628	294	584	1.65e-88	290	serine/threonine protein kinase KNS1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_013081.1	53.125	32	15	0	655	686	693	724	5.58e-04	41.2	serine/threonine protein kinase KNS1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_012394.1	27.249	378	222	6	343	684	346	706	1.86e-41	159	serine/threonine protein kinase YAK1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_013214.1	28.614	332	182	10	357	681	18	301	3.31e-31	125	mitogen-activated protein kinase HOG1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_014092.1	28.571	364	191	16	341	684	15	329	2.97e-30	120	serine/threonine/tyrosine protein kinase MCK1 [Saccharomyces cerevisiae S288c]
spo:NP_001018187.2	sce:NP_009537.1	28.319	339	183	10	362	682	13	309	2.04e-27	112	mitogen-activated serine/threonine-protein kinase FUS3 [Saccharomyces cerevisiae S288c]

2026/2/5

例) Gemini (Google)



2026/2/5

プロンプト

- 生成系AIに与える質問文を「プロンプト /prompt」と呼びます。
 - 生成系AIにプロンプトを与えることで回答が得られます。
 - よりよいプロンプトを与えることで、こちらが目的とする回答を得られる可能性が高くなります。
- プロンプト・エンジニアリング

2026/2/5

「良い」プロンプトを作るために

- ・「明確かつ具体的である」プロンプトを作るのがまず第一歩です。
- 与えられている課題を分析、分解します。
- 分解されたそれぞれについてプロンプトを作ります。
- 得られた回答が目的通りならその回答をもとにプロンプトを足していきます。

2026/2/5

例) 演習

- 与えられているのは？ → .blastoutファイル
- 最終目的は？ → awkのコマンドを出すこと
- ファイルの構成はどのようなものか
- どの部分が必要なのか(何番目のカラム？)
- 環境は
- 回答に使用して欲しいのは

以上のことを踏まえたプロンプトを作ってみる

2026/2/5

プロンプトと出力例

スペースを区切り文字とした以下のようなファイル
file.blast があります。第 1 カラムと第 2 カラムの値のみ
を抜き出してください。環境はbashで、awkコマンドを
使用してください。

```
aaaa bbbb 14 5 6 78.9 cccc  
aaab bbbc 15 6 7 89.1 cccd
```

ご指定の要件（bash環境、awk使用、第1・第2カラムの抽出）に対するコマンドは以下の通りです。

実行コマンド

Bash

```
awk '{print $1, $2}' file.blast
```

2026/2/5

プロンプトと出力例

スペースを区切り文字とした以下のようなファイル
file.blast があります。第11カラムの値が $1.0e-5$ より大きい行のみを抜き出してください。環境はbashで、awkコマンドを使用してください。

```
aaaa bbbb 14 5 6 78.9 cccc 1112.4e-100  
aaab bbbc 15 6 7 89.1 cccd 1111.5e-1
```

ご指定の条件（第11カラムが 1.0×10^{-5} より大きい）で抽出するコマンドは以下の通りです。

awk はデフォルトで `1.0e-5` のような指数表記（科学的記法）を数値として正しく認識して比較計算を行います。

コマンド

Bash

```
awk '$11 > 1.0e-5' file.blast
```

2026/2/5

プロンプトと出力例

このawkコマンドを改変します。必要なのは行すべてではなく第1、第2、第13カラムのみです。前の条件を満たし、かつ第1、第2、第13カラムのみを表示させるようにしてください。

条件（第11カラム $> 1.0e-5$ ）を維持しつつ、出力するカラムを限定するコマンドは以下の通りです。

コマンド

Bash

```
awk '$11 > 1.0e-5 { print $1, $2, $13 }' file.blast
```

2026/2/5

プロンプトの注意

- 機密性、秘匿性の高い情報を入力しない

生成系AIは、入力された情報を教育データとして学習を行っています。そのため、入力した情報は外部に漏洩する可能性があります。

利用の際には手元のデータをそのまま使用するのではなく、あたりさわりのない文書や書き換えを適宜行いましょう。

2026/2/5

プロンプトの注意

- 回答結果を検証する

生成系AIの回答は、表現や言い回しが自然であるため、正しいと感じてしまいます。しかし、情報が最新ではないことや、偏った情報が反映されてしまうこともあるなど、必ずしもその内容が「正確」とは限りません。得られた回答が正しいかどうかを検証するようにしましょう。

生成系AIをプログラムコーディングに使用する場合、一番簡単な検証は「実行してみる」ことです。

もしエラーが出たなら、そのエラーをもとにさらに生成系AIにプロンプトを与えると問題解決に近づくことが期待されます。

2026/2/5

プロンプトの注意

- 生成系AIで生成したものであると明示する

生成系AIで資料を作成する場合、
生成系AIで生成したものであるという明示をしておくとい良いでしょう。

これにより「自分個人の考え」と、「生成系AIでの回答」を別にすることができ、
論拠をはっきりさせることができます。